Software Requirements Specification

for

Crowdy

Version 3.0

Prepared by Tan Leong Jun Joseph, Teo Kok Wei Oscar, Mao Yikai, Xu Hanwen, K Eswaran

SCSJ Group 3, SC2006 Software Engineering, Nanyang Technological University

01/11/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History iii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 3
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements 6
3.1 User Interfaces 6
3.2 Hardware Interfaces 10
3.3 Software Interfaces 10
3.4 Communications Interfaces 10
4. System Features 10
4.1 Background Crowd Data 11
4.2 Map and Nearby Venues 12
4.3 Venue Detail 13
4.4 Crowd Heatmap 13
4.5 Search 14
4.6 Accounts and Saved Venues 14
4.7 Preferences 15
4.8 Friends 16
4.9 Meetup Planner 16
5. Other Nonfunctional Requirements 17
5.1 Performance Requirements 17
5.2 Safety Requirements 18
5.3 Security Requirements 18
5.4 Software Quality Attributes 18
5.5 Business Rules 19
6. Other Requirements 19
Appendix A: Glossary and Data Dictionary 20
Appendix B: Analysis Models 22
Appendix C: To Be Determined List 31
Appendix D: Changes Since Lab 1 31
Appendix E: Team Contributions 32

Revision History

Name	Date	Reason For Changes	Version
SCSJ Group 3	16/09/2026	Requirements, data dictionary, initial use case model and UI mockups (Lab 1)	1.0
SCSJ Group 3	21/10/2026	Requirements refined alongside the Lab 3 design	2.0
SCSJ Group 3	01/11/2026	Final SRS for the delivered app (Lab 5)	3.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Crowdy, version 1.0, the release delivered for the SC2006 Software Engineering project in November 2026. It covers the whole system: the Crowdy iOS app and the Crowdy backend server, together with the external data sources they depend on.

It is the final, consolidated version of the requirements first documented in Lab 1 and refined through Labs 2 to 4. Where the delivered app differs from the Lab 1 requirements, this document describes the delivered app. The main changes are listed in Appendix D.

1.2 Document Conventions

- Functional requirements are grouped into nine system features (Section 4), numbered to match the nine use case groups. Requirement REQ-4.2 is the second requirement of feature 4.
- Non-functional requirements are numbered by type: PER (performance), SAF (safety), SEC (security), QA (software quality attributes) and BR (business rules).
- "Shall" states a required capability. "Must" states a performance requirement or a constraint.
- Each feature has a priority of High, Medium or Low. Its requirements inherit that priority unless stated otherwise.
- Use cases are referred to by name and ID, for example PlanMeetup (#9-1). Their full descriptions are in the companion document SCSJ_Group3_Lab5_UseCases.
- Text shown in the app is quoted exactly, in double quotes.

1.3 Intended Audience and Reading Suggestions

- Course assessors and the lab supervisor: start with Sections 1 and 2 for the product overview, then Section 4 for what the app does.
- Developers: Sections 2.4, 2.5 and 3 for the environment and interfaces, then Section 4 and Appendix B for the design.
- Testers: Section 4 for testable functional requirements and Section 5 for measurable non-functional requirements.

Section 3.1 shows every main screen of the app. Appendix A defines the terms used throughout.

1.4 Product Scope

Crowdy is a mobile app that lets commuters, students and residents in Singapore check how crowded MRT/LRT stations and shopping malls are before heading there. It shows live station crowd levels from LTA DataMall and each mall's usual crowd for the current hour from Google

Popular Times, on one map and list. When a mall is busy, Crowdy suggests quieter malls nearby and gives directions to them. Groups of friends can ask Crowdy where to meet: it suggests malls that are fair for everyone to reach by public transport and not too crowded at the chosen time.

The objectives are to help people avoid crowds, spread visits to less busy places, and make group plans fairer. Crowdy combines three free public data sources (LTA DataMall, Google Popular Times and OneMap) in a way none of them offers on its own.

1.5 References

- SCSJ Group 3, SCSJ_Group3_Lab5_UseCases (use case model and descriptions, companion to this SRS).
- SCSJ Group 3, Lab 1 to Lab 4 deliverables, in deliverables/ of the team repository.
- Team repository: http://10.96.189.23/sc2006/scsj-h3 (GitLab) and https://github.com/josephtlj/crowdy (GitHub mirror).
- Land Transport Authority, LTA DataMall API User Guide, https://datamall.lta.gov.sg.
- Singapore Land Authority, OneMap API documentation, https://www.onemap.gov.sg/apidocs/.
- Expo, Expo SDK 58 documentation, https://docs.expo.dev.
- K. E. Wiegers, Software Requirements Specification template, 1999.

2. Overall Description

2.1 Product Perspective

Crowdy is a new, self-contained product. It is not part of a product family and does not replace an existing system. It has two parts: a mobile app (Expo / React Native) that people use, and a backend server (Node.js / Express) that gathers and merges crowd data. The app only ever talks to the backend. The backend talks to LTA DataMall, Google Maps and OneMap, and stores accounts, saved venues, friends and a shared copy of the Popular Times data in a hosted MySQL database. Figure 2.1 shows the architecture.

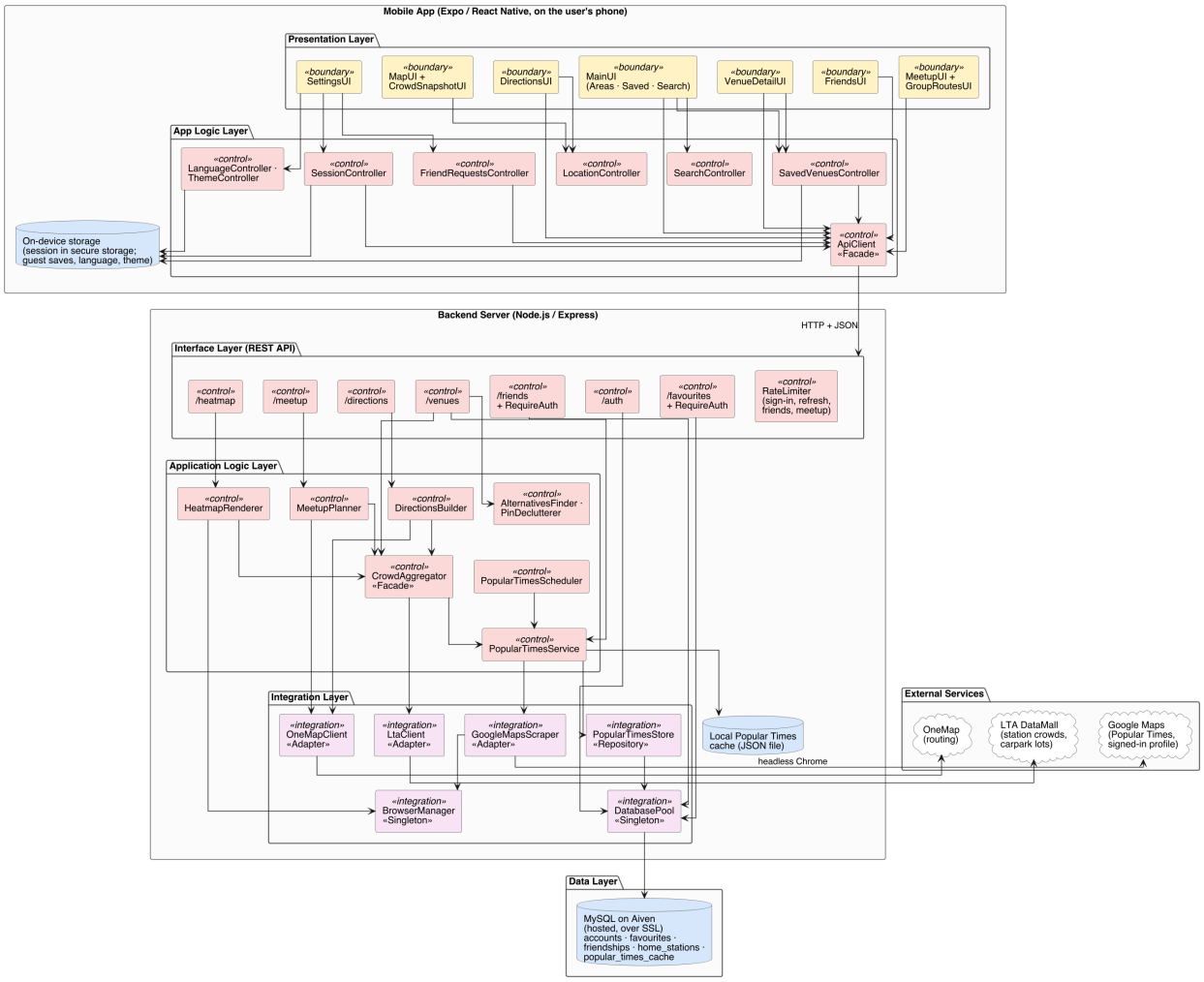


Figure 2.1. Crowdy system architecture

The app follows a boundary, control and entity split. Screens and components are boundary classes, contexts and services are control classes, and the types are entity classes. On the server, routes form the interface layer, services the application logic layer, and the clients for LTA, Google, OneMap and MySQL the integration layer.

2.2 Product Functions

Figure 2.2 shows the use case diagram. Crowdy's major functions are:

- Background crowd data: gather live station crowds from LTA DataMall and each mall's weekly Popular Times pattern, and merge them into one venue list.
- Map and nearby venues: show nearby malls and stations as crowd-coloured dots and a list, with a summary of crowds across Singapore.
- Venue detail: show a mall's crowd level, typical week, opening hours, quieter alternatives and directions.
- Crowd heatmap: show where it is busy as a coloured overlay.
- Search: find any tracked venue by name, address or postal code.
- Accounts and saved venues: sign up, log in, and save venues to the phone or an account.

- Preferences: English or Simplified Chinese, dark or light mode.
- Friends: add friends by username, answer friend requests and share a home station with friends.
- Meetup planner: suggest malls for a group to meet at, and show everyone's routes.

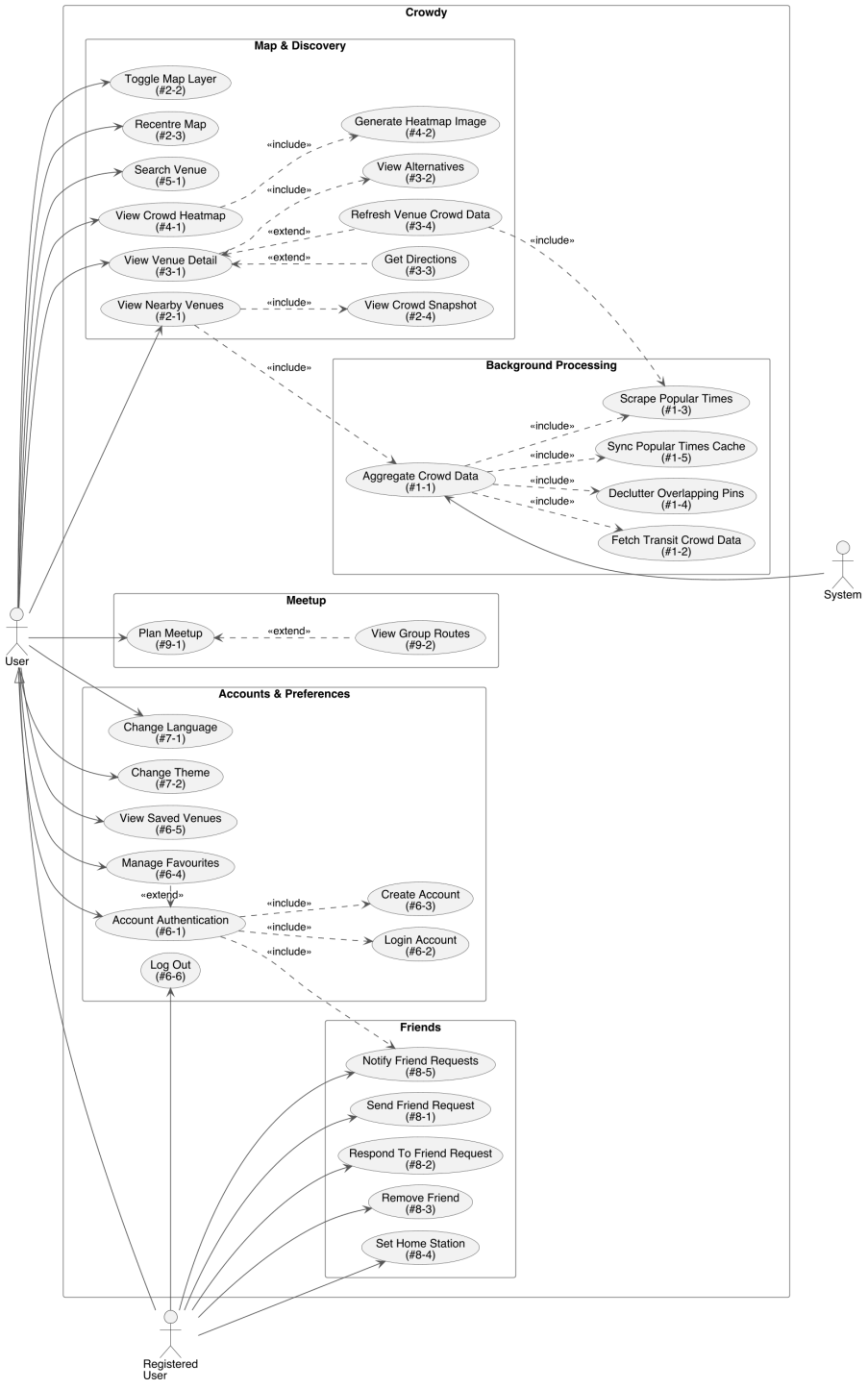


Figure 2.2. Crowdy use case diagram

In Figure 2.2 each «include» or «extend» label sits above its dashed line, or beside it where the line runs up and down. The arrow points to the use case being included or extended.

2.3 User Classes and Characteristics

User class	Characteristics
User (guest)	Anyone using the app without an account: commuters, students, residents and visitors in Singapore. Uses the map, venue pages, search, heatmap, directions and the meetup planner. Saved venues stay on the phone. Expected to use the app several times a week, often on the move. No technical knowledge assumed.
Registered User	A User who has signed in with a username and password. Can do everything a guest can, and also sync saved venues to the account, add friends, answer friend requests and set a home station.
Project team (backend operator)	Runs the backend and the one machine that collects Popular Times data. Not a user of the app's screens.

2.4 Operating Environment

- App: iPhone running Expo Go for Expo SDK 58 (React Native 0.88). Location services are used when the user allows them.
- Backend: Node.js 22.13 or later (or 24.3 or later, or 26 or later) on a computer reachable from the phone over the network. Developed and tested on macOS.
- Popular Times collection: one designated macOS machine running the backend with Google Chrome and a signed-in Google profile.
- Database: MySQL hosted on Aiven, reached over SSL.
- External services: LTA DataMall, Google Maps and OneMap, over the internet.

2.5 Design and Implementation Constraints

- The app runs in Expo Go, so it cannot use custom native modules. As a result there is no map on Android (Expo Go dropped Google Maps support) and the heatmap overlay is iOS only.
- Only free data sources are used, in line with the lab manual's guidance on API costs.
- Google offers no Popular Times API, so the data is read from Google Maps pages by a headless browser. Google only shows Popular Times to signed-in visitors, so this runs on one machine with a signed-in profile, and other backends read the shared copy from the database.
- Accounts and saved venues must be stored in MySQL, as set in the Lab 1 requirements.
- LTA DataMall and OneMap need an account key and account credentials, kept in the server's environment file and never in the repository.
- Both the app and the backend are written in TypeScript.

2.6 User Documentation

- README files in the repository: features, screenshots, architecture, setup of the app and backend, and how to run the tests.
- Short hints inside the app where a feature needs explaining, for example on the Meet up tab and the Friends screen.
- JSDoc comments on key classes and public methods in the source code.

There is no separate user manual.

2.7 Assumptions and Dependencies

- LTA DataMall's Station Crowd Density and Carpark Availability APIs stay available and keep their current format.
- Google Maps keeps showing Popular Times and opening hours to signed-in visitors in its current page layout. A layout change can stop new data being collected, although existing data keeps being served.
- OneMap's routing API stays available for public transport, walking, driving and cycling routes.
- Popular Times data is collected at a low rate, one page every 15 to 25 seconds, for a non-commercial academic project.
- Users allow location access for nearby venues and directions. Without it the map starts at a default location.
- The free tier of the Aiven MySQL service stays available.

3. External Interface Requirements

3.1 User Interfaces

Crowdy is a single-window phone app. The home screen is a map with a sheet below it that the user can drag up. The sheet has a tab bar with five tabs. Areas, Saved and Search switch the list in the sheet, while Meet up and Settings open their own screens. Every screen follows the same conventions:

- Crowd levels use one colour scale everywhere: Low green, Moderate yellow, High orange and Very High red. Closed and Unavailable venues are both shown in grey.
- Venues appear as rounded cards. A selected card has a red outline and expands in place.
- Dark mode is the default, with a light mode in Settings. All text is available in English and Simplified Chinese.
- Times are written in lowercase am/pm, for example "4:00 pm".
- Errors are shown as short plain messages on the screen concerned, never as technical codes.

Figures 3.1 to 3.18 are screenshots of the delivered app, used as its final UI mockups. They replace the prototype screenshots in the Lab 1 deliverables.

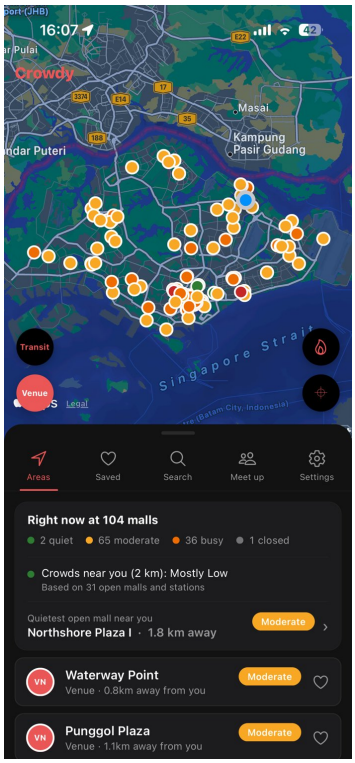


Figure 3.1. Home: map, Right now card and venue list

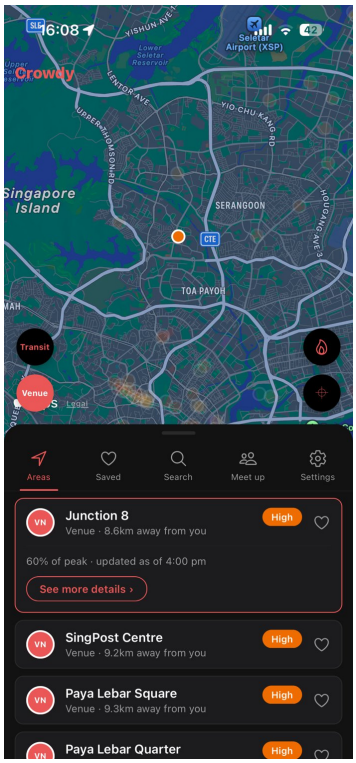


Figure 3.2. A selected venue card, expanded in place

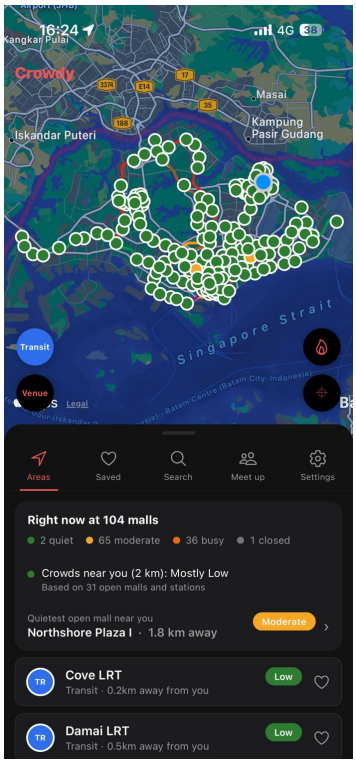


Figure 3.3. Transit layer on

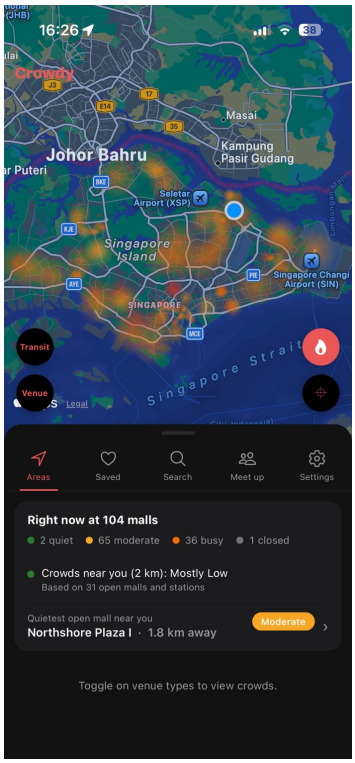


Figure 3.4. Crowd heatmap

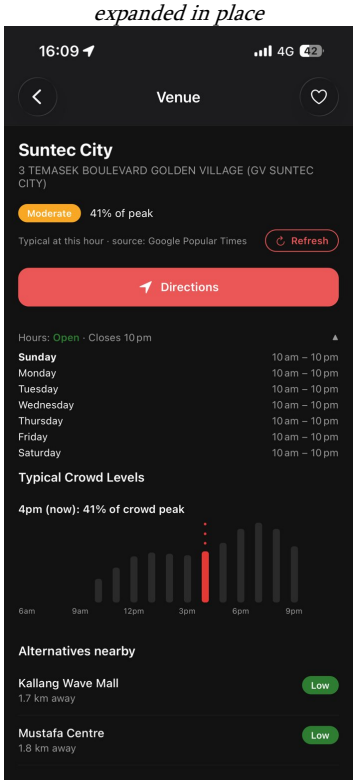


Figure 3.5. Venue page: crowd level, hours, typical crowd

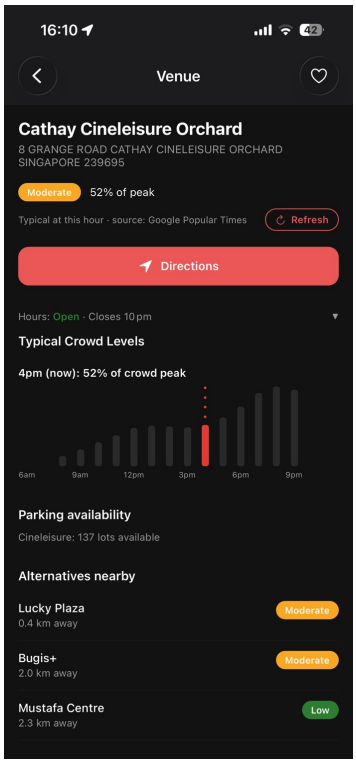


Figure 3.6. Venue page: parking and alternatives

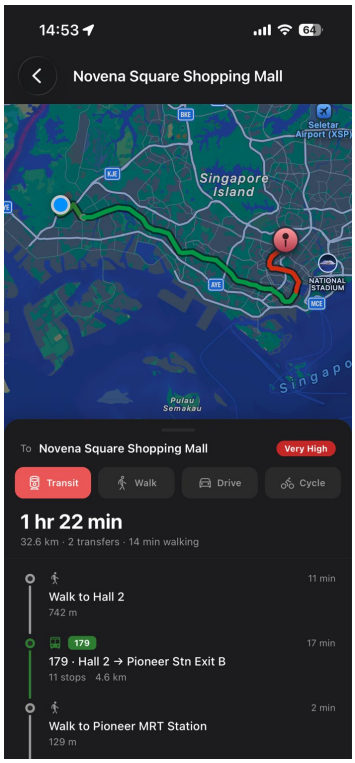


Figure 3.7. Directions: route on the map

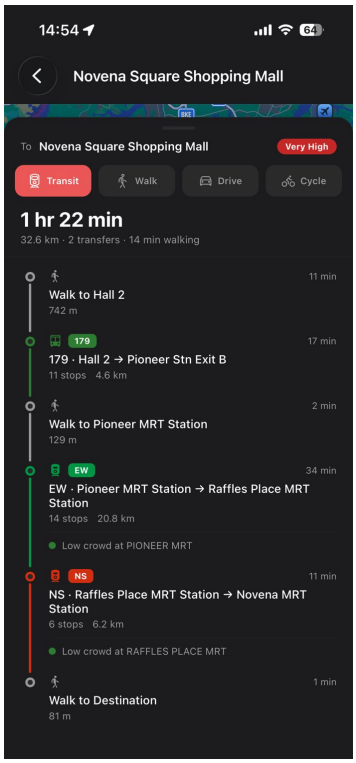


Figure 3.8. Directions: step list with station crowds

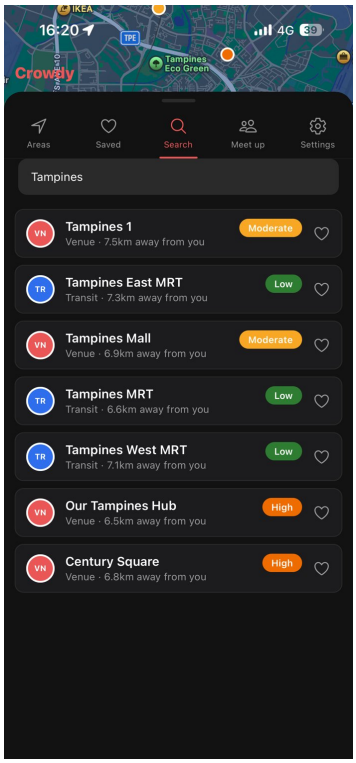


Figure 3.9. Search

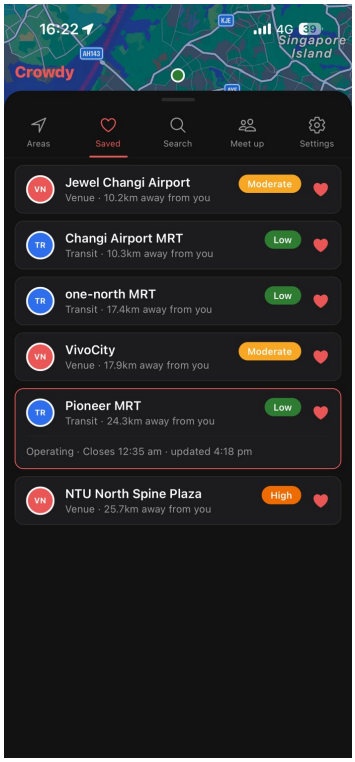


Figure 3.10. Saved venues

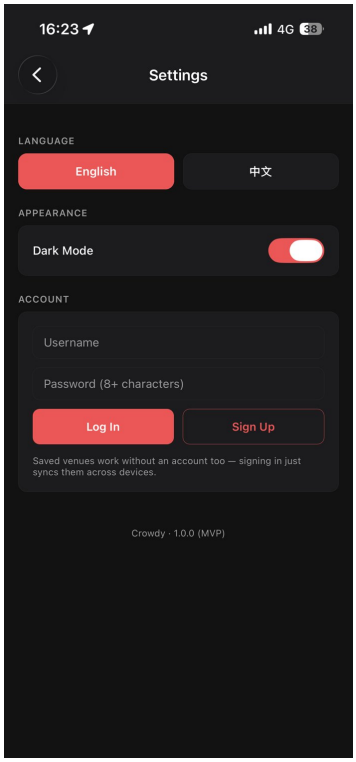


Figure 3.11. Settings

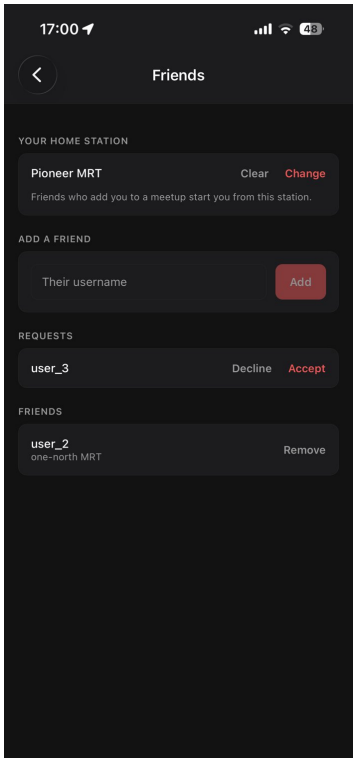


Figure 3.12. Friends

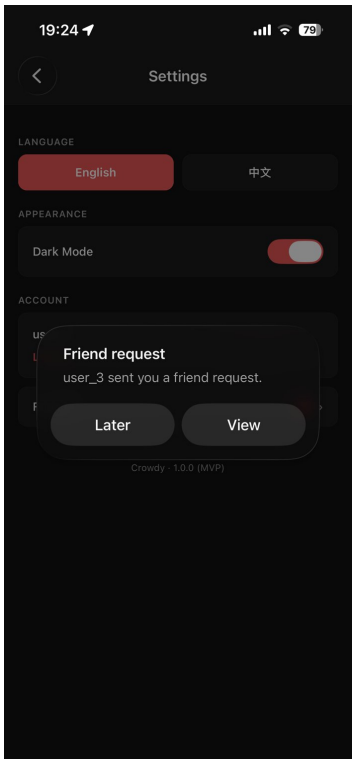


Figure 3.13. Friend request alert

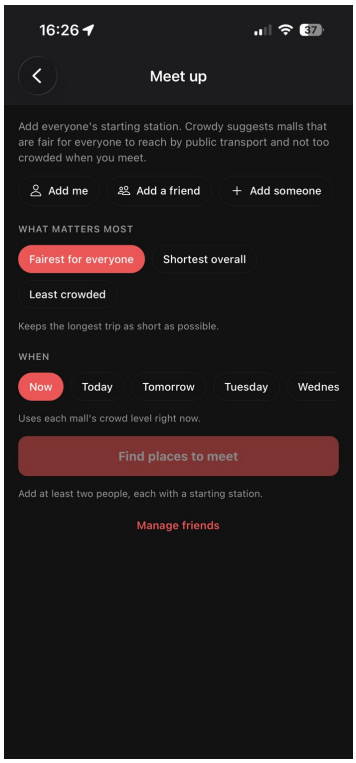


Figure 3.14. Meet up: start

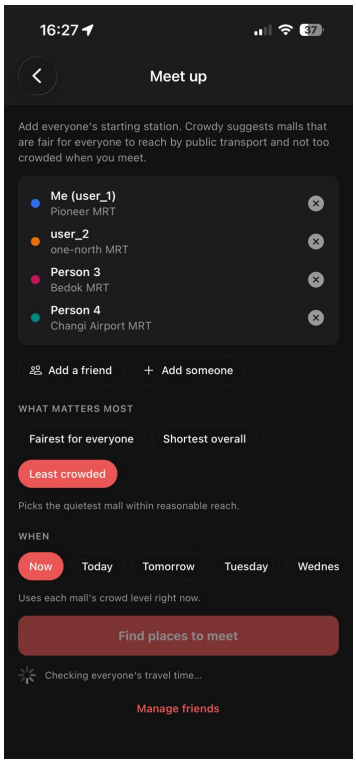


Figure 3.15. Meet up: group, priority and time

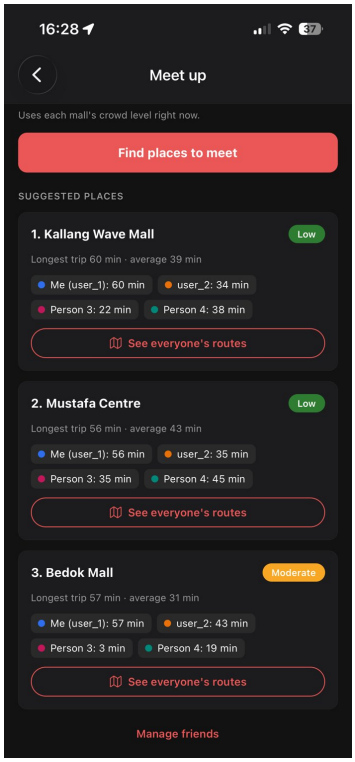


Figure 3.16. Meet up: suggested places

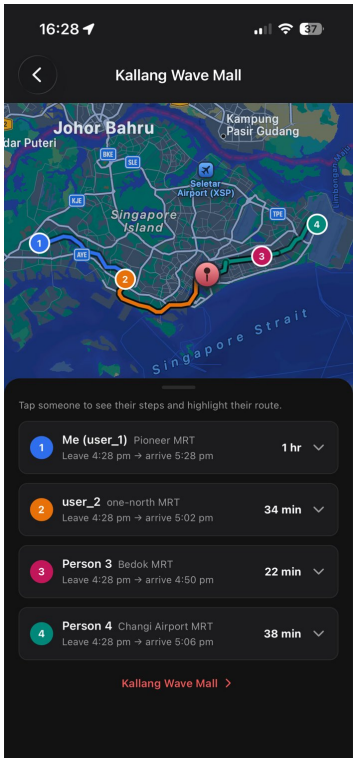


Figure 3.17. Group routes: everyone's trips

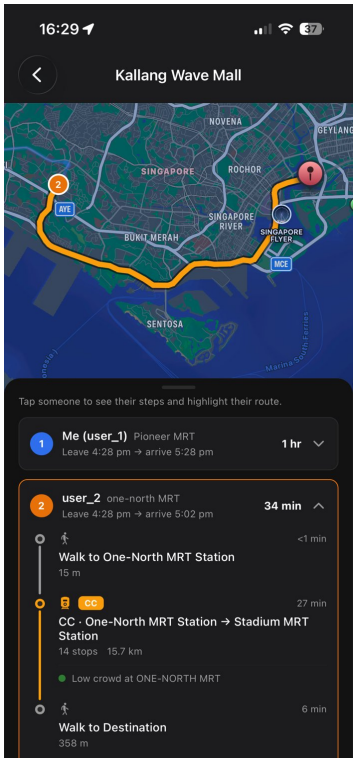


Figure 3.18. Group routes: one person's steps

3.2 Hardware Interfaces

- The app reads the phone's location through the operating system's location services, for nearby venues, directions and the Right now card.
- The app uses the touchscreen only. It needs no other device hardware.
- The backend needs no special hardware.

3.3 Software Interfaces

Component	Used for	Data exchanged
LTA DataMall: PCDRealTime	Live MRT/LRT station crowd density	Station code and crowd level (low, moderate, high), every 5 minutes
LTA DataMall: CarParkAvailabilityv2	Bonus carpark figure on mall pages	Available lots per carpark development, every minute
Google Maps (headless Chrome via Puppeteer)	Popular Times and opening hours	Hourly busyness for each day of the week, and opening hours per day
OneMap: routing API	Directions, meetup travel times and group routes	Route geometry, steps, durations and distances. Token from OneMap's auth API
MySQL on Aiven (mysql2 driver)	Persistent data	Accounts, favourites, friendships, home stations, shared Popular Times cache
Expo SDK 58 / React Native 0.88	App runtime	Expo Go on iOS
react-native-maps	Map, dots, routes, heatmap overlay	Apple Maps on iOS
expo-location, expo-secure-store, AsyncStorage	Location, session storage, preferences	GPS position, session token, guest saves, language and theme

3.4 Communications Interfaces

- The app and backend communicate over HTTP with JSON bodies through a REST API (by default on port 4000). In development the app finds the backend from the Expo development server's address, or from EXPO_PUBLIC_API_BASE_URL.
- Requests that need an account carry the session token in the Authorization header as a bearer token.
- The backend calls LTA DataMall, OneMap and Google Maps over HTTPS.
- The backend connects to MySQL over SSL, verifying the server with Aiven's CA certificate.
- Browsers may only call the backend from the local web build. The phone app is not affected by this rule.

4. System Features

Crowdy's functional requirements are organised into nine system features, one for each use case group. Each feature lists its priority, the use cases that realise it, its stimulus and response

sequences, and its functional requirements. Contributors other than the main author are named next to their use cases.

4.1 Background Crowd Data

4.1.1 Description and Priority

The backend keeps an up-to-date list of every tracked venue and its crowd level, merging live station data from LTA DataMall with each mall's weekly Popular Times pattern.

Priority: High

Use cases: AggregateCrowdData (#1-1), FetchTransitCrowdData (#1-2), ScrapePopularTimes (#1-3), DeclutterOverlappingPins (#1-4), SyncPopularTimesCache (#1-5)

4.1.2 Stimulus/Response Sequences

1. On a schedule, and when the app asks for venues, the system checks whether its station data and mall patterns are due for a refresh.

2. The system refreshes what is due and serves one merged venue list.

4.1.3 Functional Requirements

- REQ-1.1: The system shall fetch MRT/LRT station crowd levels from LTA DataMall's Station Crowd Density API.
- REQ-1.2: The system shall map each station's crowd code to Low, Moderate or High.
- REQ-1.3: The system shall mark a station Closed outside its first and last train times, whatever the raw reading.
- REQ-1.4: The system shall collect each tracked mall's weekly Popular Times pattern and opening hours from Google Maps.
- REQ-1.5: The system shall refresh a mall's Popular Times pattern when it is more than 7 days old.
- REQ-1.6: The system shall derive a mall's current crowd percentage and level from its pattern for the current day and hour.
- REQ-1.7: The system shall show a mall as Closed when its opening hours say it is closed.
- REQ-1.8: The system shall use known fixed opening hours for a mall when Google lists none.
- REQ-1.9: The system shall store every collected pattern in the shared database, and load newer patterns from it on start-up and every 15 minutes.
- REQ-1.10: The system shall fetch carpark lot availability from LTA DataMall and attach it to the matching malls.
- REQ-1.11: The system shall reposition venues that share a near-identical location so that each can be tapped separately.
- REQ-1.12: If a data source fails, the system shall keep serving its last known data for that source.
- REQ-1.13: The system shall show a mall that has never been collected as Unavailable.

4.2 Map and Nearby Venues

4.2.1 Description and Priority

The home screen shows nearby venues on a map and in a list, each with its crowd level, and a summary of crowds across Singapore.

Priority: High

Use cases: ViewNearbyVenues (#2-1), ToggleMapLayer (#2-2), RecentreMap (#2-3), ViewCrowdSnapshot (#2-4)

4.2.2 Stimulus/Response Sequences

1. The user opens the app. The system shows the map around the user's location, the venues as crowd-coloured dots, the Right now card and a list of venue cards.

2. The user pans the map. The list changes to the venues in view.

3. The user taps a card or a dot. The system expands that card and highlights the venue on the map.

4. The user taps a layer button or the recentre button. The system shows or hides that layer, or recentres the map.

4.2.3 Functional Requirements

REQ-2.1: The app shall show the user's current location on an interactive map.
REQ-2.2: The app shall show each venue as a dot coloured by its crowd level.
REQ-2.3: The app shall list the venues in the visible map area, sorted by distance from the user.
REQ-2.4: The app shall update the list as the user pans or zooms the map.
REQ-2.5: The app shall keep the list unchanged while a venue is selected, and update it when the venue is deselected.
REQ-2.6: When the user selects a venue, the app shall expand its card in place, fade the other map dots and draw the selected dot on top.
REQ-2.7: The expanded card of an open mall shall show its percentage of peak and the current hour, for example "41% of peak · updated as of 4:00 pm".
REQ-2.8: The expanded card of a closed mall shall show "0% of peak" and when it next opens.
REQ-2.9: The expanded card of a station shall show whether it is operating, its next first or last train time, and when its reading was updated.
REQ-2.10: When the user taps a dot on the map, the app shall scroll the list once to that venue's card.
REQ-2.11: The app shall let the user show or hide the Transit and Venue layers. The Venue layer shall be on and the Transit layer off by default.
REQ-2.12: The app shall let the user recentre the map on their current location.
REQ-2.13: The app shall show a Right now card with the number of malls that are quiet, moderate, busy and closed, the usual crowd level within 2 km, and the quietest open mall within 5 km.
REQ-2.14: If location is unavailable, the app shall centre the map on a default location (Marina Bay).
REQ-2.15: The app shall refresh the venue data every 2 minutes while it is open.

4.3 Venue Detail

4.3.1 Description and Priority

A mall's page shows its crowd level, typical week, opening hours, parking and quieter alternatives, with directions and a manual refresh.

Priority: High

Use cases: ViewVenueDetail (#3-1), ViewAlternatives (#3-2), GetDirections (#3-3, Teo Kok Wei Oscar), RefreshVenueCrowdData (#3-4)

4.3.2 Stimulus/Response Sequences

1. The user taps See details on a mall. The system shows the mall's page.
2. The user taps an hour on the chart. The system shows that hour's usual busyness.
3. The user taps Directions. The system shows a route from the user's location.
4. The user taps Refresh. The system fetches a fresh reading if one is due.

4.3.3 Functional Requirements

REQ-3.1: The app shall show a mall's name, address, crowd level and crowd percentage.

REQ-3.2: The app shall show a mall's crowd percentage as its typical value for the current day and hour, labelled as typical rather than live, with its data source.

REQ-3.3: The app shall show the mall's opening hours and whether it is open now, expandable to the whole week.

REQ-3.4: The app shall show the mall's typical crowd for each hour of the current day, and let the user tap an hour to see its value.

REQ-3.5: The app shall show carpark lot availability where LTA has data for the mall.

REQ-3.6: The system shall suggest up to 3 malls within 3 km of the viewed mall with a lower crowd percentage, nearest first.

REQ-3.7: The app shall show a route from the user's location to the venue by public transport, walking, driving or cycling.

REQ-3.8: For each MRT/LRT leg of a route, the app shall show the boarding station's current crowd level.

REQ-3.9: The app shall recalculate the route when the user moves more than 150 m.

REQ-3.10: The app shall let the user refresh a mall's Popular Times reading, at most once every 15 minutes per mall.

REQ-3.11: After a refresh, the app shall tell the user whether the reading was updated, already up to date, loaded from the shared copy, or could not be refreshed.

4.4 Crowd Heatmap

4.4.1 Description and Priority

An overlay that colours busy areas yellow to red so hotspots stand out.

Priority: Medium

Use cases: ViewCrowdHeatmap (#4-1), GenerateHeatmapImage (#4-2)

4.4.2 Stimulus/Response Sequences

1. The user taps the heatmap button on the map. The system shows the overlay.
2. The user taps it again. The system hides the overlay.

4.4.3 Functional Requirements

REQ-4.1: The app shall show or hide a crowd heatmap overlay on the map (iOS).
REQ-4.2: The heatmap shall colour malls and stations at Moderate crowd level or above, from yellow to red.
REQ-4.3: Quiet, closed and no-data venues shall add no colour.
REQ-4.4: The system shall generate the heatmap as one image fitted to its geographic bounds, and reuse it for 5 minutes.

4.5 Search

4.5.1 Description and Priority

Find any tracked venue, not only those on screen.

Priority: High

Use cases: SearchVenue (#5-1, K Eswaran, Mao Yikai)

4.5.2 Stimulus/Response Sequences

1. The user types on the Search tab. The system lists matching venues, best match first.
2. The user taps a result. The system expands its card.

4.5.3 Functional Requirements

REQ-5.1: The app shall search all tracked venues by name, address or postal code.
REQ-5.2: The app shall match at the start of words, accept common address abbreviations (for example "rd" for road), and forgive one wrong letter in a word.
REQ-5.3: The app shall rank results: exact postal code, postal code prefix, name prefix, name words, address words, then near matches.
REQ-5.4: The app shall show "no results" when nothing matches.

4.6 Accounts and Saved Venues

4.6.1 Description and Priority

Optional accounts keep saved venues across devices. Saving works without an account too.

Priority: Medium

Use cases: AccountAuthentication (#6-1), LoginAccount (#6-2), CreateAccount (#6-3), ManageFavourites (#6-4), ViewSavedVenues (#6-5), LogOut (#6-6)

4.6.2 Stimulus/Response Sequences

1. The user signs up or logs in from Settings. The system signs them in.
2. The user taps a heart. The system saves or removes the venue.

3. The user opens the Saved tab. The system lists the saved venues.

4.6.3 Functional Requirements

- REQ-6.1: The app shall let a user create an account with a username of 3 to 24 letters, numbers or underscores and a password of at least 8 characters.
- REQ-6.2: The system shall reject a username that is already taken.
- REQ-6.3: The app shall let a user log in with their username and password.
- REQ-6.4: The system shall show the same error for a wrong username or a wrong password.
- REQ-6.5: The app shall keep the user signed in until they log out or their session expires after 30 days.
- REQ-6.6: The app shall let the user save or remove a venue from its card or its page, whether or not they are signed in.
- REQ-6.7: When signed in, the system shall store saved venues against the account. Otherwise the app shall store them on the phone.
- REQ-6.8: When a user signs in with saved venues on the phone, the app shall ask whether to add them to the account.
- REQ-6.9: The app shall clear the phone's saved venues only after they have been added to the account.
- REQ-6.10: The app shall list the saved venues, sorted by distance, each with its current crowd level.
- REQ-6.11: The app shall let a signed-in user log out, and shall log them out automatically if the server rejects their session.

4.7 Preferences

4.7.1 Description and Priority

Language and appearance settings, remembered on the phone.

Priority: Low

Use cases: ChangeLanguage (#7-1, Mao Yikai), ChangeTheme (#7-2)

4.7.2 Stimulus/Response Sequences

1. The user picks English or 中文 in Settings. The system switches every screen's language.
2. The user switches Dark Mode in Settings. The system restyles every screen.

4.7.3 Functional Requirements

- REQ-7.1: The app shall let the user switch all of its text between English and Simplified Chinese.
- REQ-7.2: The app shall translate OneMap's turn-by-turn steps into Chinese, keeping venue, street and station names in English.
- REQ-7.3: The app shall let the user switch between dark mode and light mode, with dark mode by default.
- REQ-7.4: The app shall remember the language and theme on the phone.

4.8 Friends

4.8.1 Description and Priority

Registered users can add friends and share a home station, so friends can be added to a meetup without typing a station.

Priority: Medium

Use cases: SendFriendRequest (#8-1), RespondToFriendRequest (#8-2), RemoveFriend (#8-3), SetHomeStation (#8-4), NotifyFriendRequests (#8-5)

4.8.2 Stimulus/Response Sequences

1. The user enters a username and taps Add. The system sends a friend request.
2. The other user signs in. The system shows an alert and a red count for the request.
3. The other user taps Accept or Decline. The system adds the friend or removes the request.
4. The user picks a home station. The system saves it for their friends to use.

4.8.3 Functional Requirements

REQ-8.1: The app shall let a signed-in user send a friend request by username.
REQ-8.2: If the other user has already sent a request, sending one back shall make them friends at once.
REQ-8.3: The system shall not reveal whether a username exists when a request cannot be sent.
REQ-8.4: The app shall let a user cancel a request they sent.
REQ-8.5: The app shall let a user accept or decline each request they receive.
REQ-8.6: The app shall let a user remove a friend after confirming.
REQ-8.7: The app shall let a user set or clear a home station, chosen from the MRT/LRT stations.
REQ-8.8: The system shall show a user's home station only to their accepted friends.
REQ-8.9: When a signed-in user signs in, opens the app or returns to it, the app shall alert them once to each new friend request, with Later and View.
REQ-8.10: The app shall show the number of waiting requests on the Settings tab and on the Friends row, until each is answered.

4.9 Meetup Planner

4.9.1 Description and Priority

Suggests malls where a group can meet that are fair to reach by public transport and not too crowded, and shows everyone's routes.

Priority: High

Use cases: PlanMeetup (#9-1), ViewGroupRoutes (#9-2)

4.9.2 Stimulus/Response Sequences

1. The user adds 2 to 6 people with starting stations, picks a priority and a time, and taps Find places to meet. The system shows up to 3 malls with everyone's travel time.

2. The user taps See everyone's routes. The system shows each person's route on one map.

4.9.3 Functional Requirements

REQ-9.1: The app shall let the user add 2 to 6 people, each starting from an MRT/LRT station: themselves, a friend, or anyone else.

REQ-9.2: When a friend is added, the app shall use the friend's home station, or ask for a station if they have none.

REQ-9.3: The app shall let the user choose what matters most: Fairest for everyone (default), Shortest overall, or Least crowded.

REQ-9.4: The app shall let the user choose Now, or a day up to 7 days ahead with an hour between 7 am and 11 pm.

REQ-9.5: The system shall estimate each person's public transport time to candidate malls using OneMap.

REQ-9.6: The system shall rank malls by the chosen priority, penalising busier malls. Least crowded shall only consider malls where no one's trip is more than 20 minutes longer than the best option.

REQ-9.7: For a later time, the system shall use each mall's usual crowd at that hour and the train and bus timetable then.

REQ-9.8: The app shall show the top 3 malls, each with the longest and average trip, every person's time and the mall's crowd level.

REQ-9.9: The app shall mark an estimated travel time with ≈ when OneMap cannot plan the trip.

REQ-9.10: The app shall tell the user when every mall is closed at the chosen time, or when the chosen time has passed.

REQ-9.11: The app shall show everyone's route to a suggested mall on one map, each in that person's colour, with their leave and arrive times.

REQ-9.12: When a meeting time is chosen, the app shall let the user plan routes to arrive by that time or to leave at that time.

REQ-9.13: The app shall show live station crowd levels on route steps only when meeting now.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

PER-1: Station crowd data must be refreshed from LTA DataMall at most once every 5 minutes, matching LTA's update frequency.

PER-2: Carpark lot data must be refreshed at most once every minute.

PER-3: Each mall's Popular Times pattern must be refreshed when it is more than 7 days old, with 15 to 25 seconds between pages.

PER-4: A manual refresh of a mall must run a new collection at most once every 15 minutes per mall.

PER-5: The heatmap image must be reused for 5 minutes before it is generated again.

PER-6: A meetup search must run at most 8 route lookups at once, and reuse a repeated trip's travel time for 6 hours.

PER-7: The map and list must stay responsive while the user pans or zooms, with no visible pause while the list updates.

5.2 Safety Requirements

SAF-1: A mall's crowd figure must be labelled as its typical level for the hour ("Typical at this hour"), so users do not mistake it for a live count.

SAF-2: Live station crowd levels must only be shown on route steps for a meeting happening now, since a live reading would be misleading for a later trip.

SAF-3: The system must not store or share the user's location. It is used on the phone and sent only with the request that needs it.

5.3 Security Requirements

SEC-1: Passwords must be stored only as bcrypt hashes, never in plain text.

SEC-2: Signed-in requests must be authorised with a signed session token that expires after 30 days.

SEC-3: Saved venue and friend endpoints must reject requests without a valid session token.

SEC-4: Sign-in, manual refresh, friend requests and meetup searches must be rate limited per client.

SEC-5: Login errors must not reveal whether a username exists, and neither must failed friend requests.

SEC-6: A user's home station must only be visible to their accepted friends.

SEC-7: API keys, account credentials and the token secret must be kept in the server's environment file, outside the repository. The signed-in Google profile must be kept outside the repository.

SEC-8: The backend must connect to the database over SSL and verify the server's certificate.

SEC-9: The backend must only accept browser requests from the local web build, and must not announce its server software in responses.

5.4 Software Quality Attributes

QA-1: Reliability: if an external source is unreachable, the system must keep serving its last data for that source. A station keeps its last reading for up to 1 hour. A mall keeps its last pattern however old, since it describes a typical week.

QA-2: Availability: if the database is unreachable, everything except accounts, saved venues and friends must keep working from the backend's local cache.

QA-3: Portability: Crowdy must run on iOS. Android and Web are not supported targets because they have no map support: Expo Go has dropped Google Maps on Android, and react-native-maps has no web renderer.

QA-4: Maintainability: the app must reach the backend only through one API client, so screens do not depend on how data is fetched. Key classes and public methods must be documented with JSDoc.

QA-5: Supportability: a data source must be replaceable behind the backend without changing the app's screens.

QA-6: Testability: core logic must be covered by automated tests, including the black-box and white-box tests from Lab 4.

QA-7: Usability: errors must be shown as short plain messages, and every screen must be usable in English and Simplified Chinese.

5.5 Business Rules

BR-1: Anyone can use the map, venue pages, search, heatmap, directions, saved venues on the phone and the meetup planner without an account.

BR-2: Only signed-in users can sync saved venues to an account, add friends, answer friend requests and set a home station.

BR-3: A meetup group has 2 to 6 people, and a meeting time can be at most 7 days ahead, between 7 am and 11 pm.

BR-4: A home station is shared only with accepted friends.

BR-5: Only one designated machine collects Popular Times data. Every other backend reads the shared copy.

6. Other Requirements

6.1 Database Requirements

Persistent data is kept in MySQL in five tables: accounts, favourites, friendships, home_stations and popular_times_cache. The accounts and favourites tables are created by the setup script (server/scripts/init-db.ts). The friendships, home_stations and popular_times_cache tables are created automatically by the backend if they are missing. Deleting an account deletes its favourites, friendships and home station.

6.2 Internationalisation Requirements

All app text exists in English and Simplified Chinese. Dates and times follow the chosen language, for example "4:00 pm" in English and "下午4:00" in Chinese. Times use Singapore time.

6.3 Legal Requirements

Personal data must be handled in line with Singapore's Personal Data Protection Act (PDPA). Crowdy stores only a username, a hashed password, saved venues, friendships and an optional home station. It does not store email addresses, names or locations.

Appendix A: Glossary and Data Dictionary

Glossary

Term	Meaning
Crowd level	A venue's crowdedness as one of Low, Moderate, High, Very High, Closed or Unavailable.
Crowd percentage (% of peak)	A mall's busyness for an hour, relative to its own busiest hour of the week (100%).
Popular Times	Google's chart of how busy a place usually is for each hour of each day of the week.
Typical week / pattern	The full week of Popular Times data collected for one mall.
Home station	The MRT/LRT station a Registered User usually starts from, shared with accepted friends.
Friend request	An invitation from one account to another to become friends. Pending until accepted or declined.
Layer	A group of map dots that can be shown or hidden: Transit (stations) or Venue (malls).
Right now card	The summary card at the top of the Areas list.
Guest saves	Venues saved on the phone while not signed in.
LTA DataMall	The Land Transport Authority's open data service.
OneMap	The Singapore Land Authority's map and routing service.
Expo Go	The app used to run Crowdy on a phone during development and the demo.
JWT	JSON Web Token, the signed session token issued at sign-in.
PDPA	Singapore's Personal Data Protection Act.

Data Dictionary

Venue

Attribute	Type	Description
id	String	Unique identifier, e.g. "ion-orchard"
name	String	Display name
category	Enum	Transit (MRT/LRT station) or Venue (mall)
address, postalCode	String	Address, and its six-digit postal code where known
lat, lng	Float	Coordinates (WGS84)
crowdPercent	Integer	0 to 100, percentage of the mall's peak. Not used for stations
crowdLevel	Enum	Low, Moderate, High, Very High, Closed or Unavailable
source	Enum	LTA or GooglePopularTimes
lastUpdated	DateTime	When the reading was taken (station) or the pattern collected (mall)
hours	List	Opening hours (malls) or first and last train times (stations) per day
hourly	List	The typical crowd for each hour of today (malls)
carparks	List	Available lots per LTA carpark development (malls, optional)
distanceKm	Float	Distance from the user, computed when needed

Station

Attribute	Type	Description
code	String	Official LTA station code, e.g. "NS25"
id, name	String	Internal identifier and official name
mode	Enum	MRT or LRT
lat, lng	Float	Coordinates from LTA's geospatial data
address	String	Display address
hours	List	First and last train time for each day of the week

Account

Attribute	Type	Description
id	Integer	Unique identifier
username	String	3 to 24 letters, numbers or underscores, unique
passwordHash	String	bcrypt hash of the password
createdAt	DateTime	When the account was created

Favourite

Attribute	Type	Description
accountId	Integer	The account that saved the venue
venueId	String	The saved venue
createdAt	DateTime	When it was saved

Friendship

Attribute	Type	Description
requesterId, addresseeId	Integer	The account that sent the request and the one that received it
status	Enum	pending or accepted
createdAt	DateTime	When the request was sent

HomeStation

Attribute	Type	Description
accountId	Integer	The account
stationId	String	The chosen MRT/LRT station
updatedAt	DateTime	When it was last set

PopularTimesCache

Attribute	Type	Description
venueId	String	The mall
result	JSON	The typical week (hourly busyness per day) and opening hours
scrapedAt	DateTime	When the pattern was collected

MeetupParticipant

Attribute	Type	Description
stationId	String	Starting MRT/LRT station
accountId	Integer	Set for the user and friends, empty for others (not stored)

MapRegion

Attribute	Type	Description
latitude, longitude	Float	Centre of the visible map area
latitudeDelta, longitudeDelta	Float	Span (zoom level) of the visible map area

Relationships

- A Venue of category Transit corresponds to exactly one Station.
- An Account has zero or many Favourites. Each Favourite references one Venue.
- An Account sends and receives zero or many Friendships. Each Friendship links two different Accounts.
- An Account has zero or one HomeStation, which references one Station.
- A mall Venue has zero or one PopularTimesCache entry.
- Guest saves are a list of venue ids stored on the phone, not in the database. On sign-in they can be added to the Account as Favourites.
- The user's location is used only to query and sort venues and is never stored.

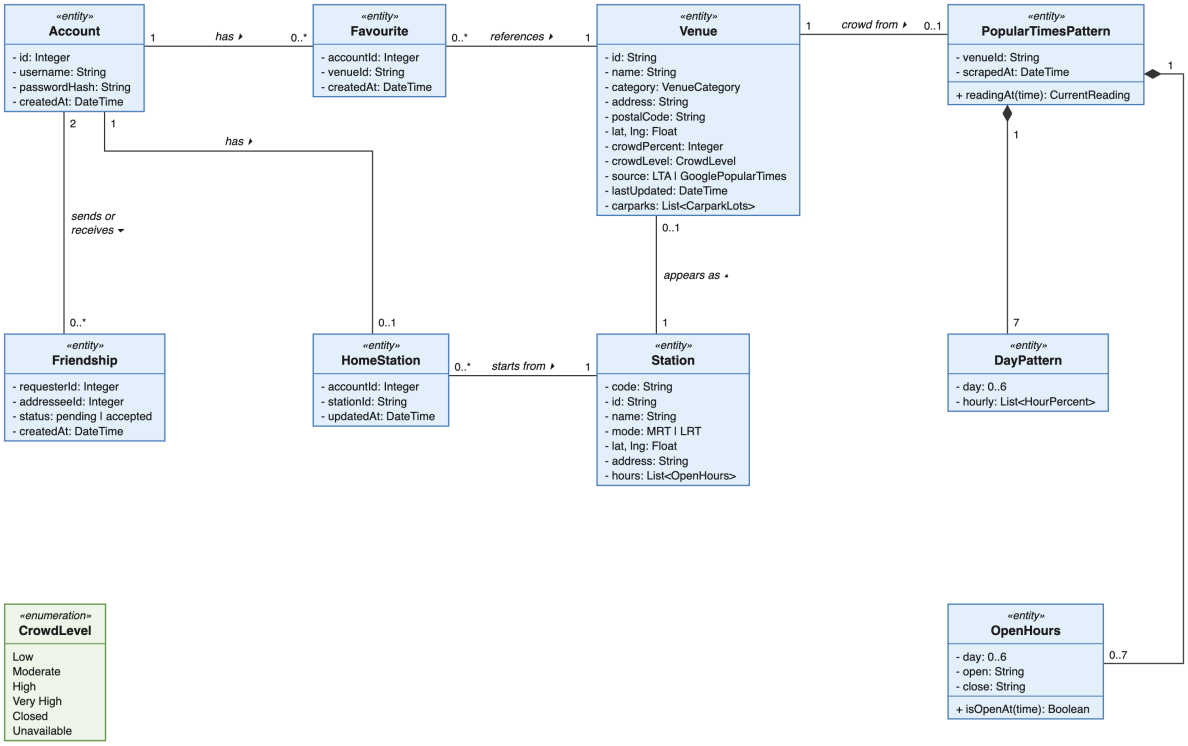
Appendix B: Analysis Models

This appendix holds Crowdy's design models: the class diagrams and the UI dialog map. The system architecture is shown in Figure 2.1 and the use case diagram in Figure 2.2. Full-resolution copies of every diagram are in SCSJ_Group3_Lab5_Diagrams/.

Class Diagrams

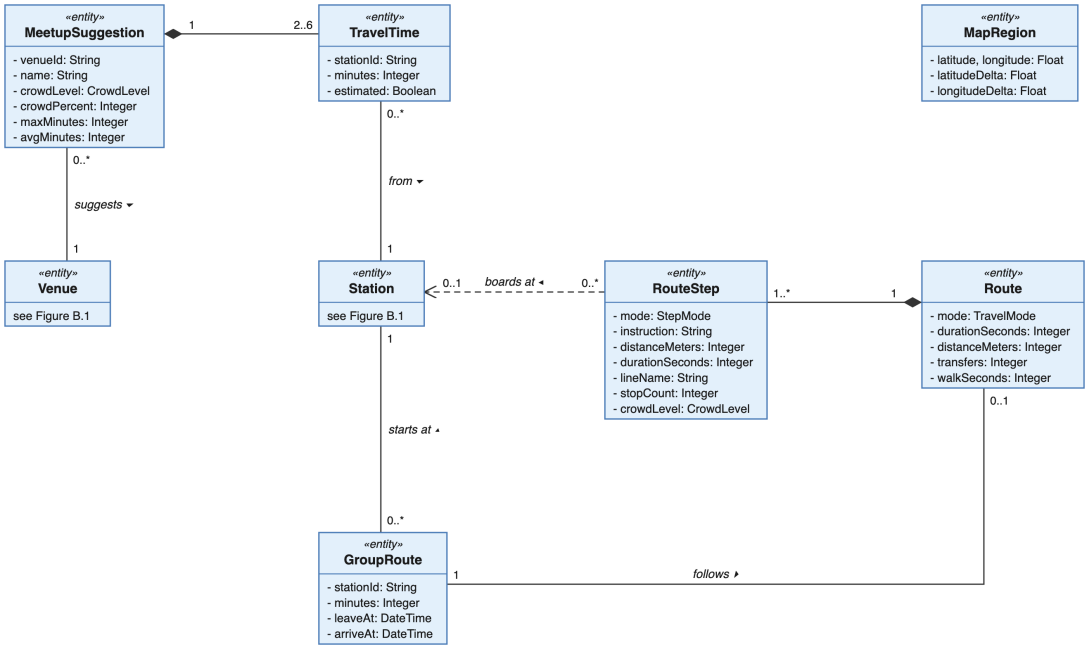
The class diagrams follow the boundary, control and entity split. Figures B.1 and B.2 show the entity classes, Figure B.3 the app's boundary and control classes, and Figures B.4 and B.5 the server's classes by layer. Figure 2.1 (system architecture) gives the overview of how they fit together. Each figure shows the key attributes and operations of its classes. Minor helpers are left out.

Design patterns used: ApiClient and CrowdAggregator are Facades over many calls. LtaClient, GoogleMapsScraper and OneMapClient are Adapters that turn each outside service into Crowdy's own types. BrowserManager and DatabasePool are Singletons shared by every request. PopularTimesStore is a Repository over the database. The four app controllers marked Observer notify the screens that use them when their state changes.



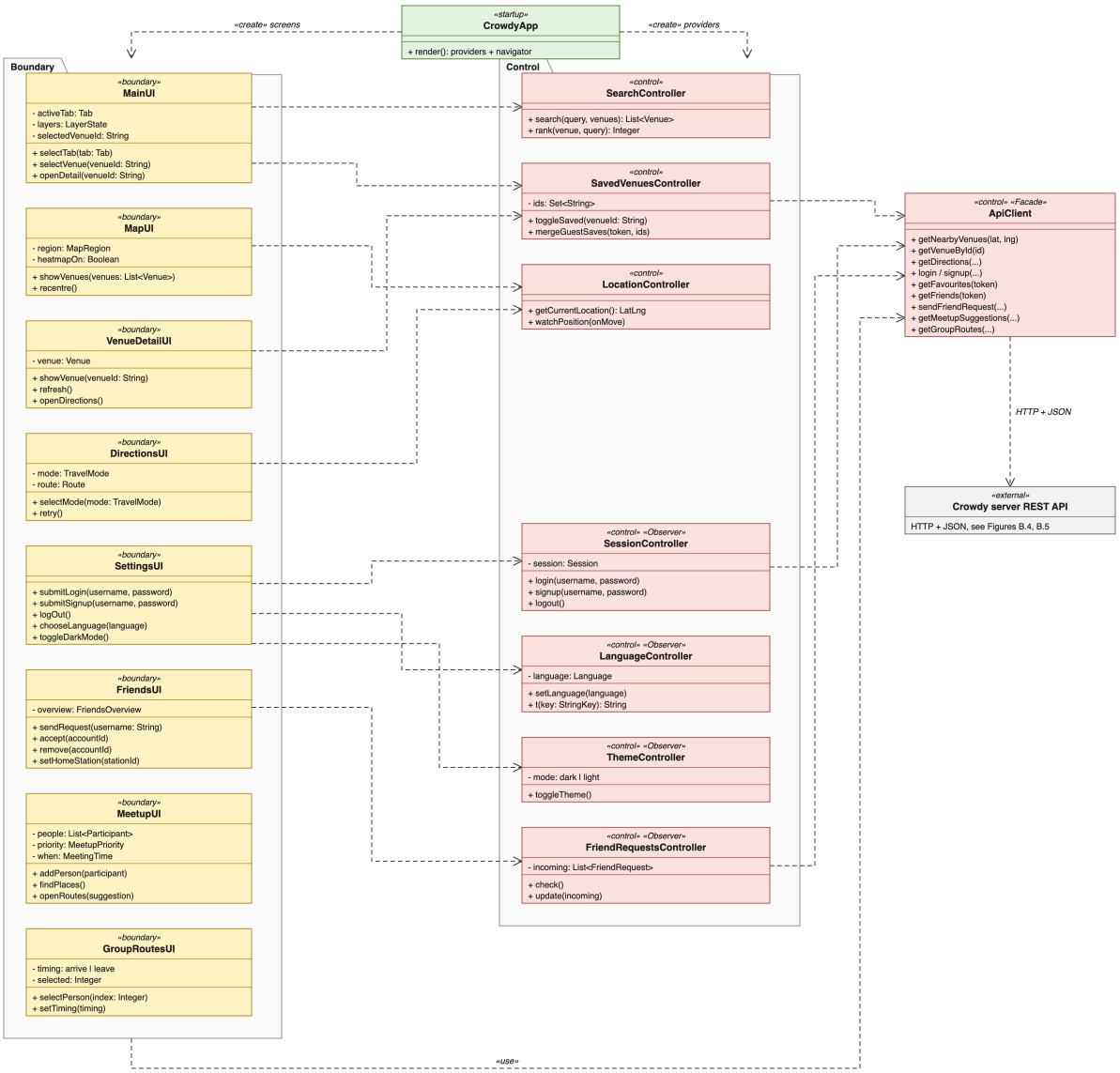
Each association shows its multiplicities, and its label has a small triangle pointing the way it reads (Account has ▸ Favourite). A filled diamond marks composition and a dashed arrow a dependency.

Figure B.1. Entity classes: accounts and crowd data



Each association shows its multiplicities, and its label has a small triangle pointing the way it reads (Account has ▸ Favourite). A filled diamond marks composition and a dashed arrow a dependency.

Figure B.2. Entity classes: routes and meetups



Dashed arrows are dependencies (the class at the tail calls the one at the head). Every screen in the Boundary package fetches data through ApiClient.

Figure B.3. App design classes

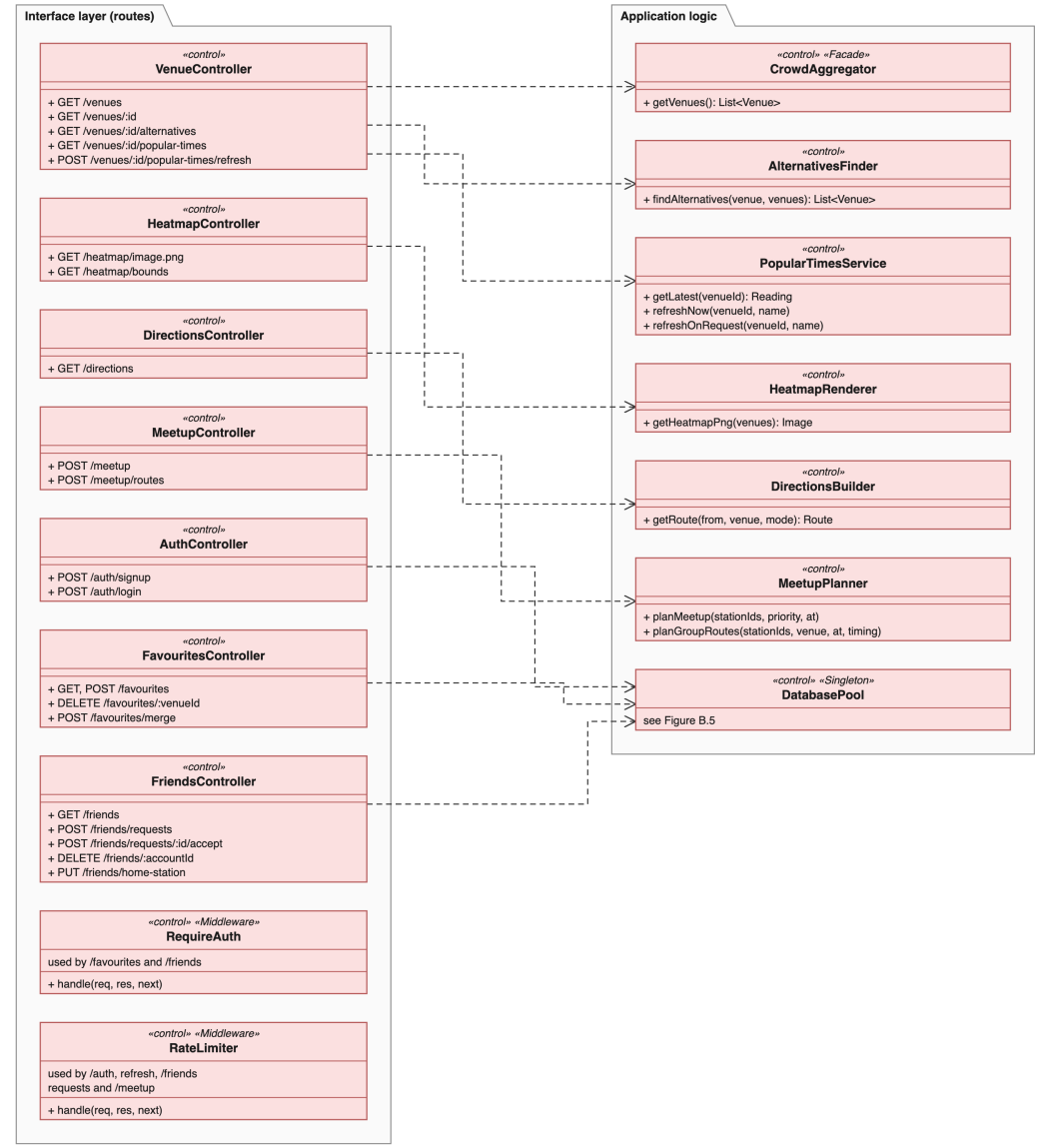


Figure B.4. Server design classes: interface layer and what it calls

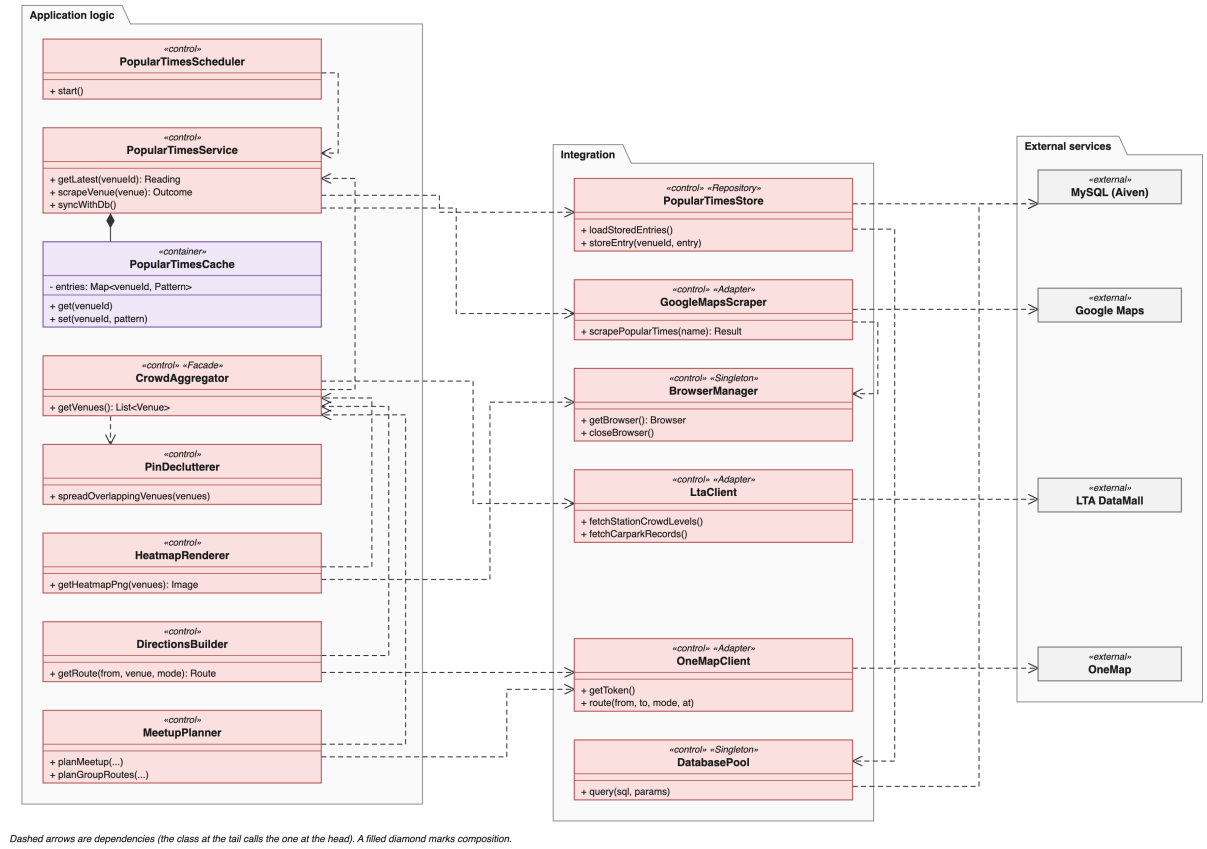


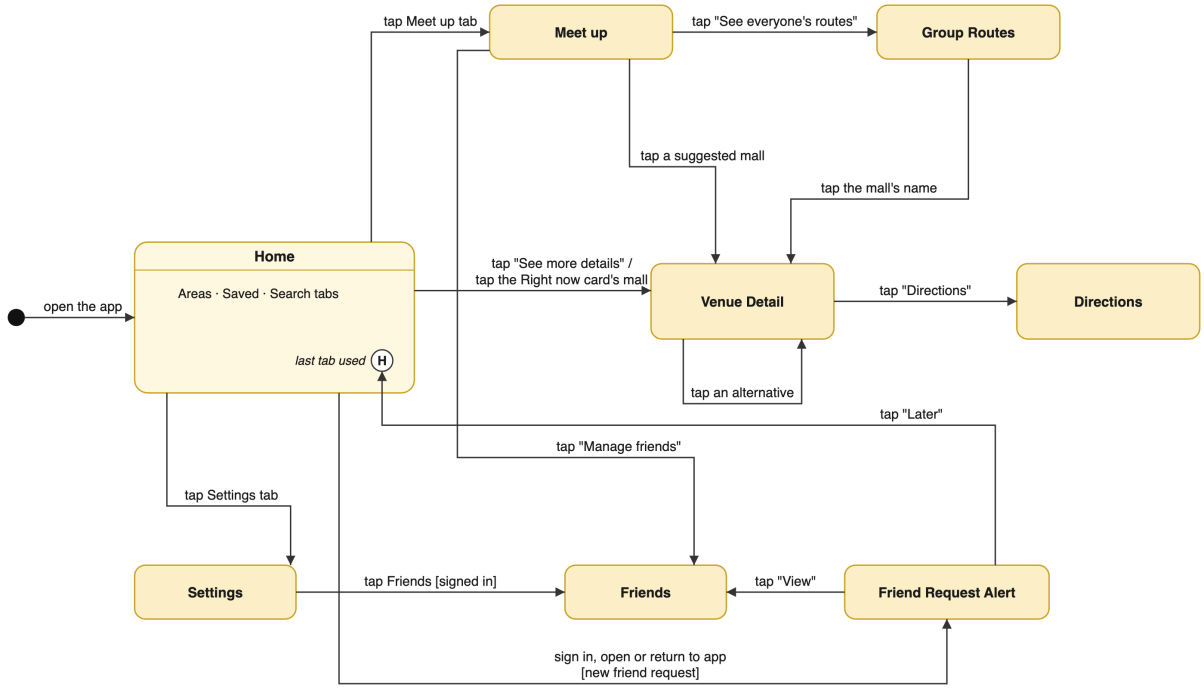
Figure B.5. Server design classes: application logic and integration

Dialog Map

The dialog map is drawn at two levels. Figure B.6 shows every screen and how the user moves between them. Figures B.7 to B.10 show the states inside the screens that have them. Each figure is a separate state machine with exactly one initial state (the black dot): in Figure B.6 it is the app opening, and in Figures B.7 to B.10 it is the user arriving on that screen through the transition shown in Figure B.6.

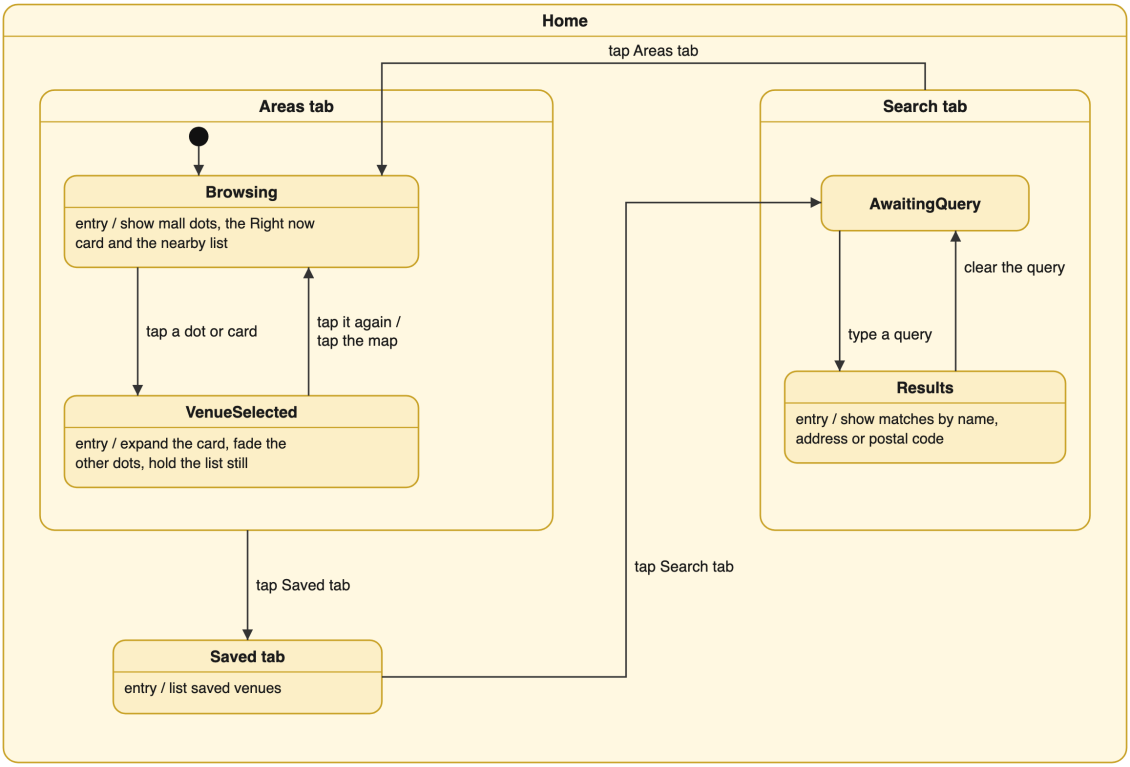
Every screen except Home has a back button that returns to the screen it was opened from. Going back to Home returns to the tab the user was last on, shown by the history state (H, labelled "last tab used"). The Home screen is the map with the Areas, Saved and Search tabs in the sheet below it: switching between these tabs only changes the list, so they are states inside Home (Figure B.7). Meet up and Settings sit in the same tab bar but open their own full screens with a back button, so they are separate states in Figure B.6. The friend request alert appears on top of the screen the user is on.

In Figure B.6 each transition's label sits above its line. In Figures B.7 to B.10 each label sits to the right of its line, or above it where the line is horizontal.



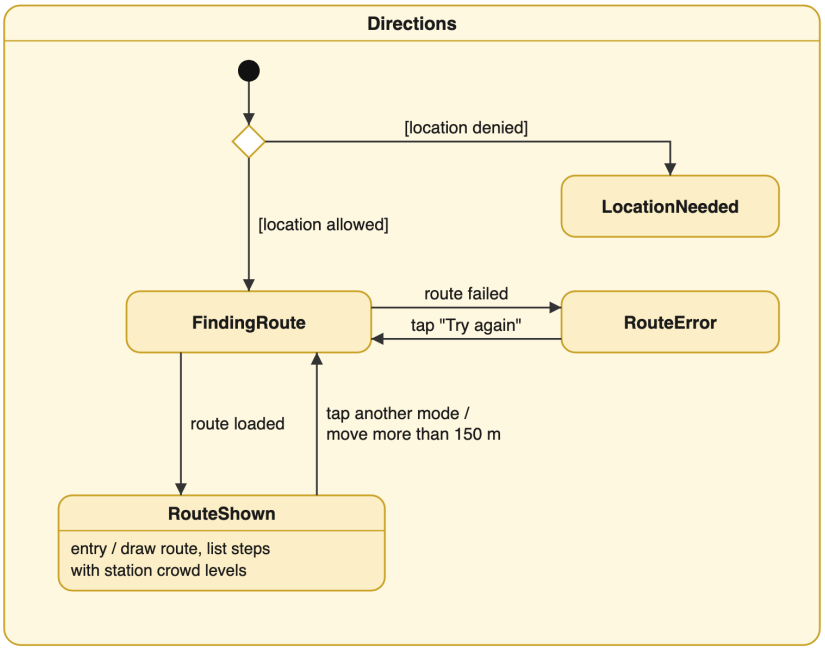
Each label sits above its line. Every screen except Home also has a back button that returns to the screen it was opened from. Going back to Home returns to the last tab used (H).

Figure B.6. Dialog map: screens and navigation



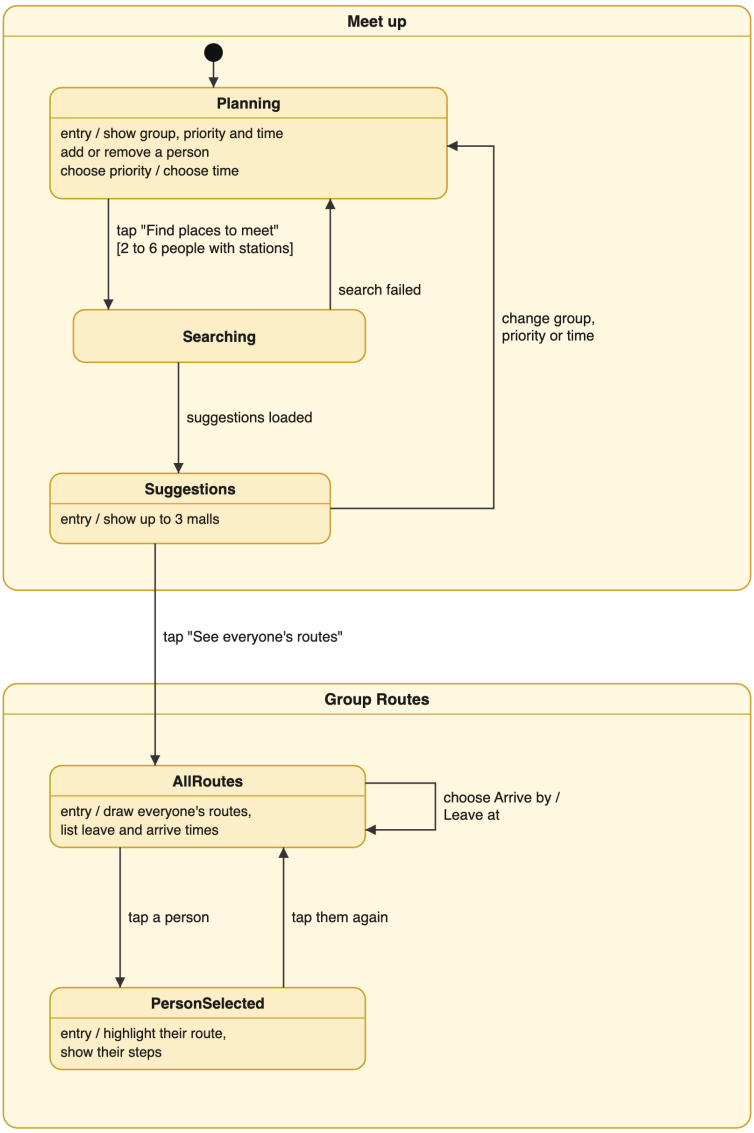
Each label sits to the right of its line, or above it where the line is horizontal.
Meet up and Settings are in the same tab bar but open their own screens, shown in Figure B.6.

Figure B.7. Dialog map: states of the Home screen



Each label sits to the right of its line, or above it where the line is horizontal.

Figure B.8. Dialog map: states of the Directions screen



Each label sits to the right of its line, or above it where the line is horizontal.

Figure B.9. Dialog map: states of the Meet up and Group Routes screens

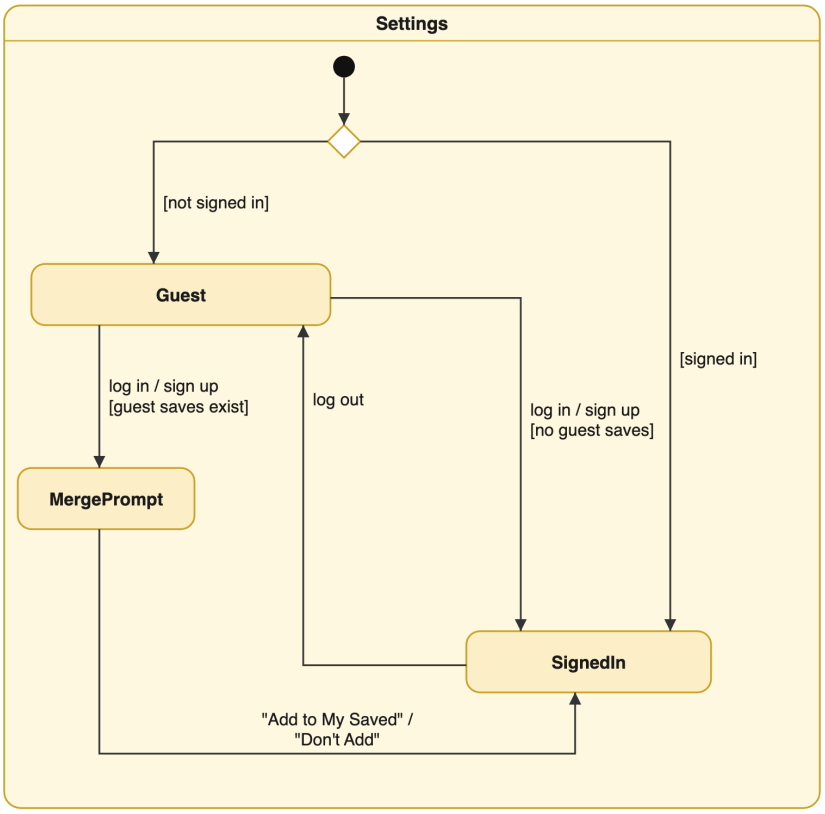


Figure B.10. Dialog map: states of the Settings screen

Appendix C: To Be Determined List

There are no to-be-determined items in this version.

Appendix D: Changes Since Lab 1

The requirements changed as the project progressed. The main changes from the Lab 1 deliverables are:

Area	Lab 1	Delivered app
Crowd data sources	Official government APIs only. Mall crowds estimated from carpark lot availability, with carpark history stored	Stations from LTA DataMall. Malls from Google Popular Times. Carpark lots shown as a bonus figure only, with no history stored
Venues covered	MRT, LRT, malls and attractions	MRT/LRT stations and 104 validated malls. Attractions dropped
Map markers	Category-labelled pins, clustered when zoomed out	Crowd-coloured dots in two layers (Transit, Venue). Clustering dropped, overlapping venues spread apart instead
Venue list	Follows the visible map area	Same, but holds still while a venue is selected. Selected cards expand in place
Venue detail	Last updated time and source	"Typical at this hour" for malls, plus weekly chart, opening hours, parking, directions and manual refresh
Accounts	Email and password. Guest saves merged automatically	Username and password. The user is asked before guest saves are added
New features	None	Crowd heatmap, search, Right now card, directions, English/Chinese, dark/light mode, friends with request alerts, meetup planner and group routes
Portability	iOS, Android and Web with parity	iOS only. Android and Web have no map support
Data dictionary	Account (email), Favourite (account or device), CarparkHistory	Account (username), Favourite (account only), Friendship, HomeStation, PopularTimesCache. CarparkHistory removed
Use cases	Initial model (Lab 1)	31 use cases in 9 groups. ClusterMapMarkers removed, friends, meetup, refresh and theme added (see SCSJ_Group3_Lab5_UseCases)

Appendix E: Team Contributions

Member	Contributions
Tan Leong Jun Joseph	System design and most of the implementation: crowd data collection, map and list, venue pages, heatmap, accounts, friends, meetup planner and group routes. Testing (Lab 4). Lab 3 to Lab 5 documentation and demo script.
Teo Kok Wei, Oscar	In-app directions (GetDirections #3-3). Account use case descriptions (#6-1 to #6-6).
Mao Yikai	Search by address and postal code with typo tolerance (SearchVenue #5-1), English and Chinese language support (ChangeLanguage #7-1), guest-save merge fix, with tests.
Xu Hanwen	ViewAlternatives use case (#3-2). Uploaded the Lab 1 and Lab 2 deliverables.
K Eswaran	SearchVenue use case description (#5-1).